Estructura de carpetas

Vitrina Regional — App COIL UNIMINUTO + AUNAR Villavicencio

Flutter · Clean Architecture + Feature-First · flutter_bloc · go_router · Firebase
Generado el 3 de noviembre de 2026

Este proyecto Flutter sigue **Clean Architecture** organizada **por funcionalidades** (*feature-first*). Cada funcionalidad es independiente y se divide internamente en tres capas: `data`, `domain` y `presentation`. El código compartido vive en `lib/core` y el cableado de la app en `lib/app`.

1. Raíz del proyecto

```
vitrina-regional/
├── lib/            # Código fuente de la app (Dart)
├── test/           # Pruebas unitarias y de widgets (espejo de lib/)
├── docs/           # Documentación: plan de tareas y decisiones
├── android/ ios/   # Proyectos nativos por plataforma
├── web/ macos/ linux/ windows/
├── .github/        # Workflows de CI (GitHub Actions)
├── .vscode/ .fvm/  # Config del editor y versión de Flutter (FVM)
├── pubspec.yaml      # Dependencias y metadatos del paquete
├── analysis_options.yaml  # Reglas de lint
├── firebase.json / firestore.rules / storage.rules  # Config Firebase
├── README.md / CONTRIBUTING.md
└── .gitignore / .metadata
```

2. Carpeta `lib/` — tres bloques

Carpeta	Rol
`lib/app/`	Arranque y cableado: tema, router, inyección de dependencias (DI), widget raíz.
`lib/core/`	Código transversal reutilizable por todas las features: errores, utilidades, widgets base, contrato de caso de uso.
`lib/features/`	Las funcionalidades del negocio, cada una autocontenida con sus 3 capas.
`lib/main.dart`	Punto de entrada: inicializa Firebase y lanza la app.
`lib/firebase_options.dart`	Config generada por FlutterFire (+ `.example` como plantilla).

lib/app/ — cableado de la aplicación

```
lib/app/
├── app.dart                    # Widget raíz (MaterialApp.router)
├── di/ app_providers.dart      # Proveedores e inyección de dependencias
├── router/
│   ├── app_router.dart         # Configuración de go_router
│   ├── app_routes.dart         # Constantes de rutas
│   └── go_router_refresh_stream.dart  # Refresco por cambios de auth
└── theme/ app_theme.dart       # Tema visual (colores, tipografía)
```

lib/core/ — compartido

```
lib/core/
├── bloc/   app_bloc_observer.dart   # Observador global de BLoC (logs)
├── error/  exceptions.dart, failures.dart  # Excepciones y fallos tipados
├── usecase/ usecase.dart            # Contrato base de casos de uso
├── utils/  formatters.dart, validators.dart  # Formato y validación
└── widgets/ app_network_image.dart, app_text_field.dart,
             primary_button.dart, state_views.dart  # Widgets reutilizables
```

3. Anatomía de una feature (las 3 capas)

Toda feature en `lib/features/<nombre>/` repite la misma estructura de Clean Architecture. La dependencia fluye hacia adentro: **presentation** → **domain** ← **data**.

domain/ — el corazón, sin dependencias de Flutter ni Firebase.

- `entities/` — objetos de negocio puros.
- `repositories/` — contratos (interfaces abstractas).
- `usecases/` — acciones de negocio, una clase por acción.

data/ — implementa los contratos del dominio.

- `datasources/` — orígenes de datos (remoto/Firebase y mock).
- `models/` — DTOs con serialización (fromJson/toJson).
- `repositories/` — implementación concreta del repositorio.

presentation/ — la interfaz de usuario y el estado.

- `bloc/` — gestión de estado por eventos (BLoC).
- `cubit/` — estado más simple (Cubit).
- `pages/` — pantallas completas.
- `widgets/` — componentes visuales de la feature.

Ejemplo: feature `emprendimientos` (completa, de referencia)

```
lib/features/emprendimientos/
├── data/
│   ├── datasources/ emprendimiento_remote_datasource.dart, ..._mock_datasource.dart
│   ├── models/      emprendimiento_model.dart, producto_model.dart
│   └── repositories/ emprendimiento_repository_impl.dart
├── domain/
│   ├── entities/    categoria, emprendimiento, producto, ubicacion
│   ├── repositories/ emprendimiento_repository.dart
│   └── usecases/    create / update / get_by_id / get_by_owner /
│                  get / search_by_categoria / validar
└── presentation/
    ├── bloc/   emprendimientos_bloc / _event / _state
    ├── cubit/  detalle / filtro_categoria / mi_emprendimiento
    ├── pages/  home_page, detalle_page, mi_emprendimiento_page
    └── widgets/ categoria_chips, emprendimiento_card, emprendimiento_form
```

4. Las 5 features del proyecto

Feature	Qué hace	Estado
`auth`	Login, registro y sesión de usuario (Firebase Auth).	Completa (referencia)
`emprendimientos`	CRUD y listado de emprendimientos, categorías, productos.	Completa (referencia)
`mapa`	Geolocalización y ordenar emprendimientos por cercanía.	Estructura + TODOs
`resenas`	Reseñas y calificaciones en tiempo real.	Estructura + TODOs
`imagenes`	Seleccionar y subir imágenes a Firebase Storage.	Estructura + TODOs

Nota: `mapa` e `imagenes` no tienen `models/` porque trabajan con entidades simples; `imagenes` y `mapa` usan solo `cubit/` (sin bloc) en presentation. La estructura se adapta a la complejidad de cada feature.

5. Carpeta `test/` — espejo de `lib/`

Las pruebas replican la estructura de `lib/` para que cada archivo probado sea fácil de localizar. Además hay un `test/helpers/` con utilidades compartidas.

```
test/
├── app/      app_smoke_test.dart, router/ app_router_test.dart
├── core/     utils/ validators_test.dart
├── helpers/  fixtures.dart          # Datos de prueba reutilizables
└── features/
    ├── auth/            data, presentation (bloc, cubit, pages)
    ├── emprendimientos/ data, domain (usecases), presentation (bloc, cubit)
    ├── mapa/            data, domain (usecases), presentation (cubit)
    ├── resenas/         data, domain (usecases), presentation (bloc)
    └── imagenes/        data, presentation (cubit)
```

6. Carpeta `docs/`

Archivo	Contenido
`docs/PLAN_DE_TAREAS.md`	Plan de los 4 encuentros COIL: tareas, puntos, criterios de aceptación, pistas y rúbrica.
`docs/DECISIONS.md`	Registro de decisiones técnicas (ADR).

7. Resumen del flujo de dependencias

presentation (UI, BLoC/Cubit) → **domain** (entities, usecases, contratos) ← **data** (datasources, models, impl)

La capa `domain` no depende de nada externo. `presentation` y `data` dependen de `domain`, nunca al revés. La inyección de dependencias en `lib/app/di/` conecta las implementaciones concretas de `data` con los contratos que usa `presentation`.